Chapter 2
Software Process Models and Agility

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Processes

- A software process (also knows as software methodology) is a set of related activities that leads to the production of the software.
- These activities may involve the development of the software from the scratch, or, modifying an existing system.
- A structured set of activities required to develop a software system
  - Specification
  - Design and Implementation
  - Validation
  - Evolution
- A software process model is an abstract representation of a process.
- It presents a description of a process from some particular perspective.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- A process also includes the process description, which includes:
  - **Products**: The outcomes of the an activity. For example, the outcome of architectural design maybe a model for the software architecture.
  - **Roles**: The responsibilities of the people involved in the process. For example, the project manager, programmer, etc.
  - **Pre and post conditions**: The conditions that must be true before and after an activity. For example, the pre condition of the architectural design is the requirements have been approved by the customer, while the post condition is the condition to be satisfied after the activity.-

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Process Model

- A software process model is a simplified representation of a software process.
- A software process model is an abstraction of the software development process.
- A process model for software engineering is chosen based on:
  - the nature of project and application
  - The methods and tools to be used
  - The controls and deliverables that are required
- Process model may include:
  - Activities which are part of software process
  - Software product
  - Role of people involved in software engineering
- The goal of software process model is to **provide guidance for systematically coordinating and controlling the tasks that must be performed in order to achieve end products and product objectives.**

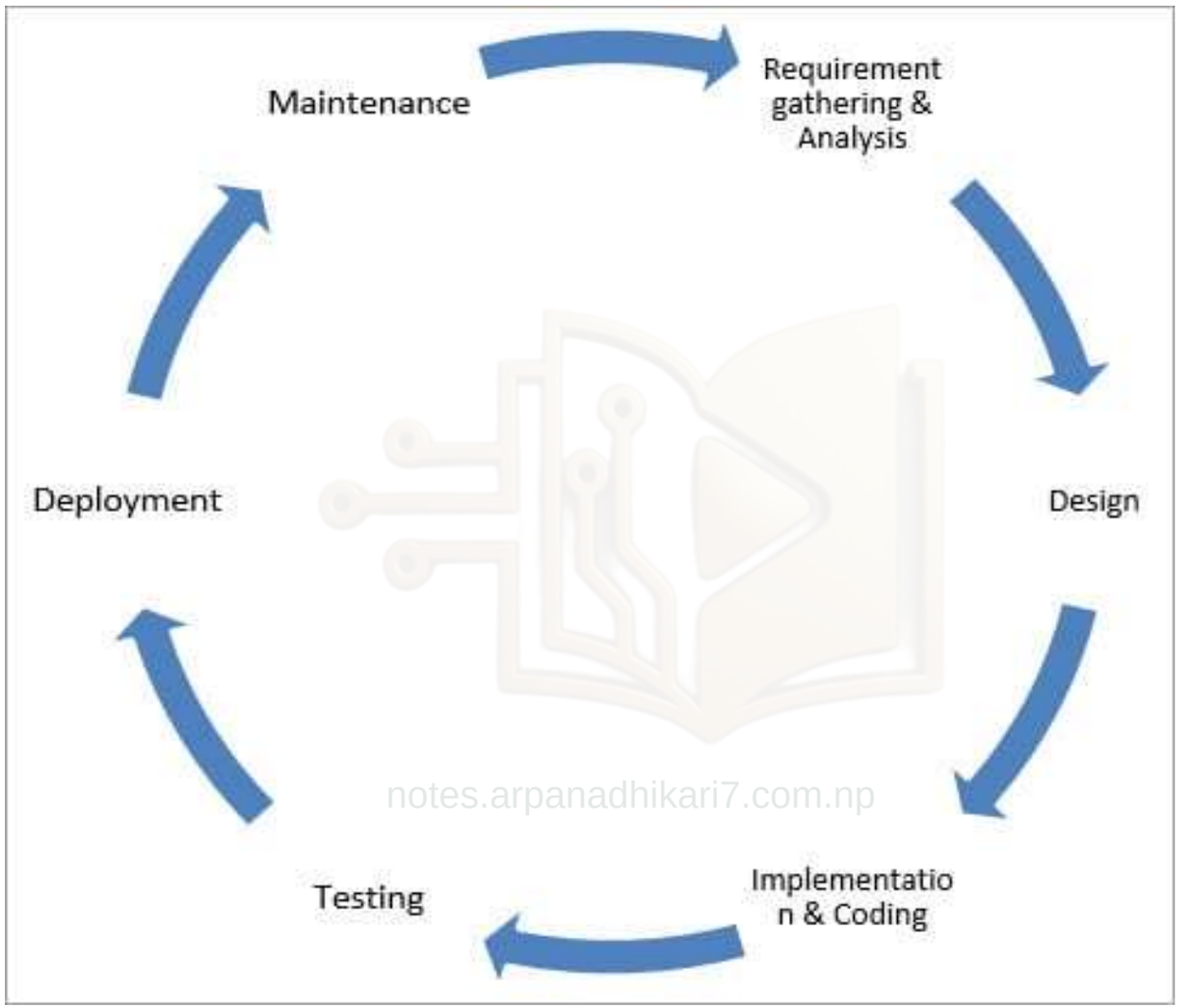
Available on notes.arpanadhikari7.com.np

Software Development Life Cycle(SDLC)

- SDLC is a process which defines the various stages involved in the development of software for delivering a high-quality product.
- SDLC stages cover the complete life cycle of a software i.e. from inception to retirement of the product.
- Purpose of SDLC is to deliver a high-quality product which is as per the customer's requirement.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Development Life Cycle(SDLC)

Requirement Gathering and Analysis

- This is the foundational phase of the Software Development Life Cycle (SDLC), where stakeholders collaborate to identify and document the requirements of the software.
- Activities include conducting client meetings, interviews, and feasibility studies to ensure technical, economic, and operational viability.
- The output of this phase is the **Software Requirement Specification (SRS)** document, which serves as a blueprint for the entire project.
- **Software Requirement Specification (SRS)** is a complete specification and description of requirements of the software that need to be fulfilled for the successful development of the software system. These requirements can be functional as well as non-functional depending upon the type of requirement.
- **SRS (Software Requirement Specification)** is a document in which all the user requirements are mentioned in a structural manner. SRS is a kind of agreement between the customer and the company, which contains complete information about the requirement desired by the customer and the product made by the company.
- For instance, an e-commerce application might require features like user accounts, product browsing, and secure payment options.
- This phase ensures a clear understanding of what the software should achieve and establishes a mutual agreement between stakeholders.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Development Life Cycle(SDLC)

Design

- The design phase transforms the requirements outlined in the SRS into a structured plan or blueprint for the software.
- This phase involves two levels: **High-Level Design (HLD)** focuses on the system architecture, data flow, and module interactions, while **Low-Level Design (LLD)** delves into the specifics like database schemas and algorithms.
- The output is detailed design documents that guide developers during implementation.
- For instance, designing the architecture of an e-commerce platform might include modules for user authentication, product catalog, and payment processing.

notes.arpanadhikari7.com.np

Software Development Life Cycle(SDLC)

Implementation and Coding

- In this phase, the design is translated into actual code using programming languages, frameworks, and tools.
- Developers write, compile, and debug the code, following predefined coding standards. Activities include version control and peer reviews to ensure quality.
- This phase results in a functional software system that aligns with the design and requirements.
- For example, coding the user authentication, shopping cart functionality, and payment gateway for an e-commerce site are part of this phase.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Development Life Cycle(SDLC)

Testing

- The testing phase ensures the developed software functions as intended and is free from defects.
- Various testing levels, such as unit testing, integration testing, system testing, and acceptance testing, are performed to validate the system against the requirements.
- Specialized tests like performance, usability, and security testing are also conducted.
- Bugs identified during testing are fixed before deployment.
- For instance, testing an e-commerce platform might involve verifying the checkout process, payment gateway functionality, and user experience across devices.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Development Life Cycle(SDLC)

Deployment

- In this phase, the software is delivered to the production environment and made available to end users.
- Deployment includes configuring the system, integrating it with existing setups, and conducting user training.
- Proper documentation is also provided to aid users and administrators.
- The outcome is a live, operational software product.
- For example, deploying an e-commerce application would involve hosting it on a cloud server, configuring the domain, and ensuring it is accessible to users.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Software Development Life Cycle(SDLC)

Maintenance

- The maintenance phase ensures the software remains functional, secure, and up-to-date after deployment.
- It involves fixing bugs, patching vulnerabilities, and adapting the software to evolving requirements or environments.
- Regular updates might include adding new features or optimizing performance.
- For example, an e-commerce site may require updates to support new payment methods or address feedback from users, ensuring continuous satisfaction and reliability.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

General Software Process Models

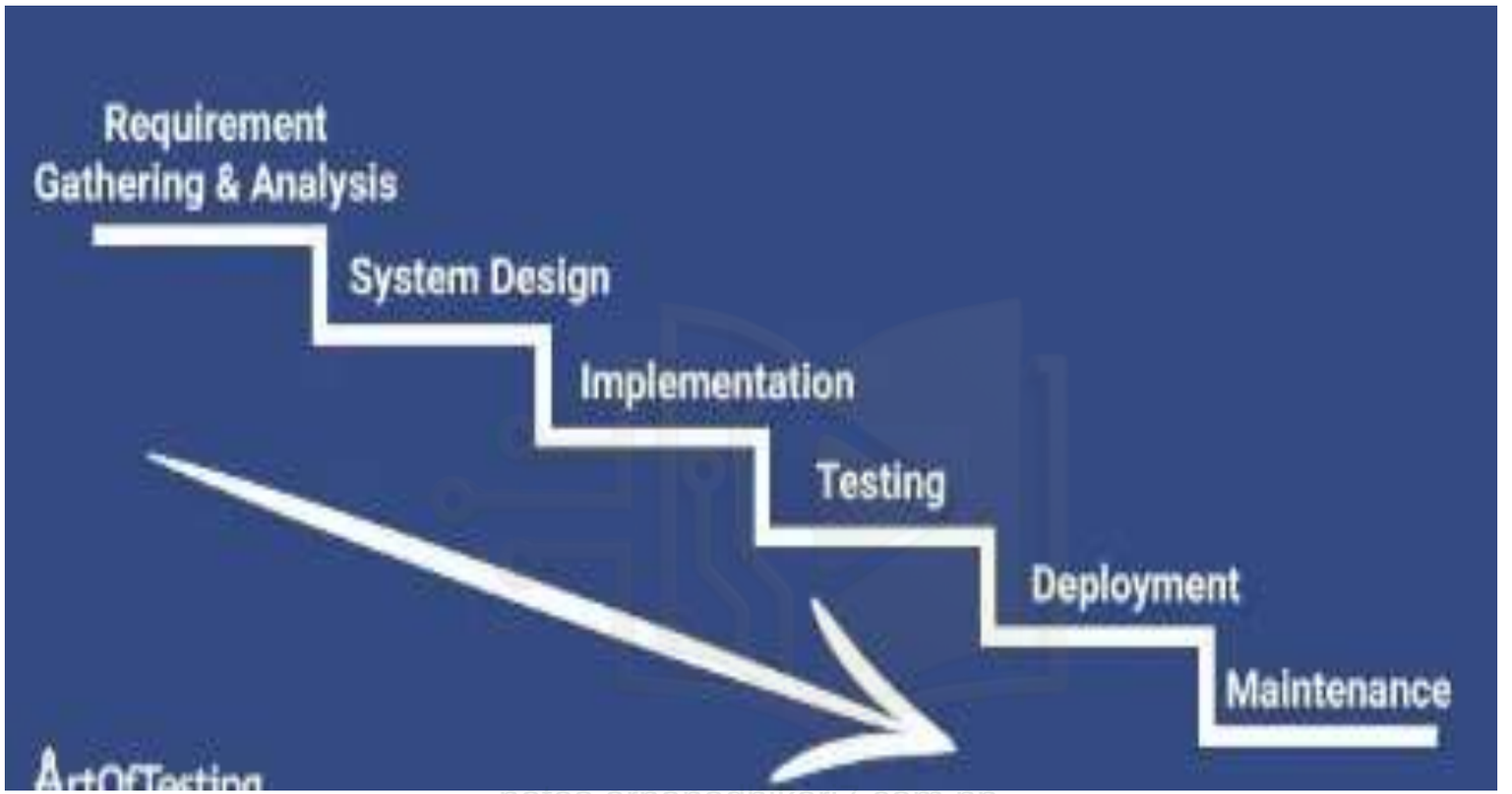
1. Waterfall
2. Incremental
3. Prototyping
4. Iterative
5. Spiral
6. Rapid Application Development
7. Aspect Oriented Software Engineering

notes.arpanadhikari7.com.np

Waterfall Model

- The Waterfall Model was the first Process Model to be introduced
- It is also referred to as a **linear-sequential life cycle model**.
- The waterfall model emphasizes a logical progression of steps.
- In this model, the whole process of software development is divided into separate phases and typically, the outcome of one phase acts as input for next phase.
- All the phases in this model are cascaded to each other in which progress is seen as flowing steadily downwards (like a waterfall) through the phases.
- The next phase is started only after the defined set of goals are achieved for previous phase and it is signed off, so the name "Waterfall Model".
- This means that any phase in the development process begins only if the previous phase is complete.

Available on notes.arpanadhikari7.com.np



notes.arpanadhikari7.com.np

Figure: Waterfall Model

Available on notes.arpanadhikari7.com.np

- The sequential phases of waterfall model are:

a. Requirement Gathering and analysis

- It is the first phase in the development cycle which involves which involves capturing all the possible requirements and documenting in a requirement specification document.
- Also the captured requirements are analyzed and communicated with the developing team.

b. System Design

- The requirement specifications from first phase are studied in this phase and the system design is prepared.
- This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.

Available on notes.arpanadhikari7.com.np

c. Implementation

- The output of the system design phase is fed as input to this phase where the system is first developed in small programs called units.
- Each unit is developed as well as tested separately for its functionality.

d. Integration and Testing

- In this phase all the units are tested referred as unit testing and finally are integrated into a system.
- Again after the integration is completed the whole system is tested for faults and failures.
- This phase involves both te functional and non-functional testing.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

e. Deployment of system

- Once all the testing is done the system is now ready to be deployed.
- The product is deployed in the customer environment or released into the market.

f. Maintenance

- There are some issues which come up in the client environment.
- To fix those issues, patches(set of changes to a computer program) are released. Also to enhance the product some better versions are released.
- Maintenance is done to deliver these changes in the customer environment.

notes.arpanadhikari7.com.np

All these phases are cascaded to each other in which progress is seen as flowing steadily downwards (like a waterfall) through the phases. The next phase is started only after the defined set of goals are achieved for previous phase. In this model phases do not overlap.

Available on notes.arpanadhikari7.com.np

When is waterfall model appropriate?

Waterfall model is appropriate when:

- Product definition is stable.
- Technology is understood and is not dynamic.
- There are no ambiguous requirements i.e. when the requirements are clear, well documented and fixed.
- Sufficient resources with required expertise are available to support the product.
- The project is short.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Advantages of Waterfall Model

Some of the major advantages of the Waterfall Model are as follows:

- The waterfall model progresses through easily understandable and explainable phases and thus it is easy to use.
- Easy to manage due to the rigidity of the model. Each phase has specific deliverables and a review process.
- Phases are processed and completed one at a time.
- Works well for smaller projects where requirements are very well understood.
- In this model, phases are processed and completed one at a time and they do not overlap.
- It is easy to arrange tasks in waterfall model.
- Process and results are well documented.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Disadvantages of Waterfall Model

- It does not allow much reflection or revision. Once an application is in the testing stage, it is very difficult to go back and change something that was not well-documented or thought upon in the concept stage.
- No working software is produced until late during the life cycle.
- High amounts of risk and uncertainty. Not suitable for projects whose requirements change frequently.
- Not a good model for complex and object-oriented projects. Poor model for long and ongoing projects..
- It is difficult to measure progress within stages.
- Cannot accommodate changing requirements.
- Adjusting scope during the life cycle can end a project.

Available on notes.arpanadhikari7.com.np

Why waterfall model failed?

a. **Overlapping:** The waterfall model has lacked an overlapping among phase. The waterfall model recommends that new phase can start only after the completion of the previous phase. But in real projects, this can't be maintained.

b. **Single way mechanism:** This model is just like the one-way street. Once a phase is completed and next phase has started then there is no way to go back on the previous phase.

c. **Not Flexible:** Difficult to accommodate change requests. The waterfall model assumes that all the customer requirements can be completely and correctly defined at the beginning of the project, but actually customers' requirements keep on changing with time.

d. **Lack of interaction:** The waterfall model has lacked interaction among phase. Feedback is not taken during development. After a development process starts, changes can not accommodate easily.

Available on notes.arpanadhikari7.com.np

Evolutionary Model

- Evolutionary Model are iterative.
- Once the requirements are analyzed, they pass through a series of iterations till the complete software is developed.
- After each release, based on the review given by the reviewers, further iterations are performed.
- It involves more user interaction in every iteration, and thereby increasing reliability.
- The iterative lifecycle is based on the successive enlargement and refinement of a system through multiple iterations, with cyclic feedback and adaptation.
- **The result of each iteration is executable but incomplete system** i.e. it is not ready deliver into production.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

➢There are two types of evolutionary development:

- Exploratory development
  - Start with requirements that are well defined
  - Add new features when customers propose new requirements
- Throw-away prototyping
  - Objective is to understand customer's requirements (i.e. they often don't know what they want, hence poor requirements to start
  - Use means such as prototyping to focus on poorly understood requirements, redefine requirements as you progress

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

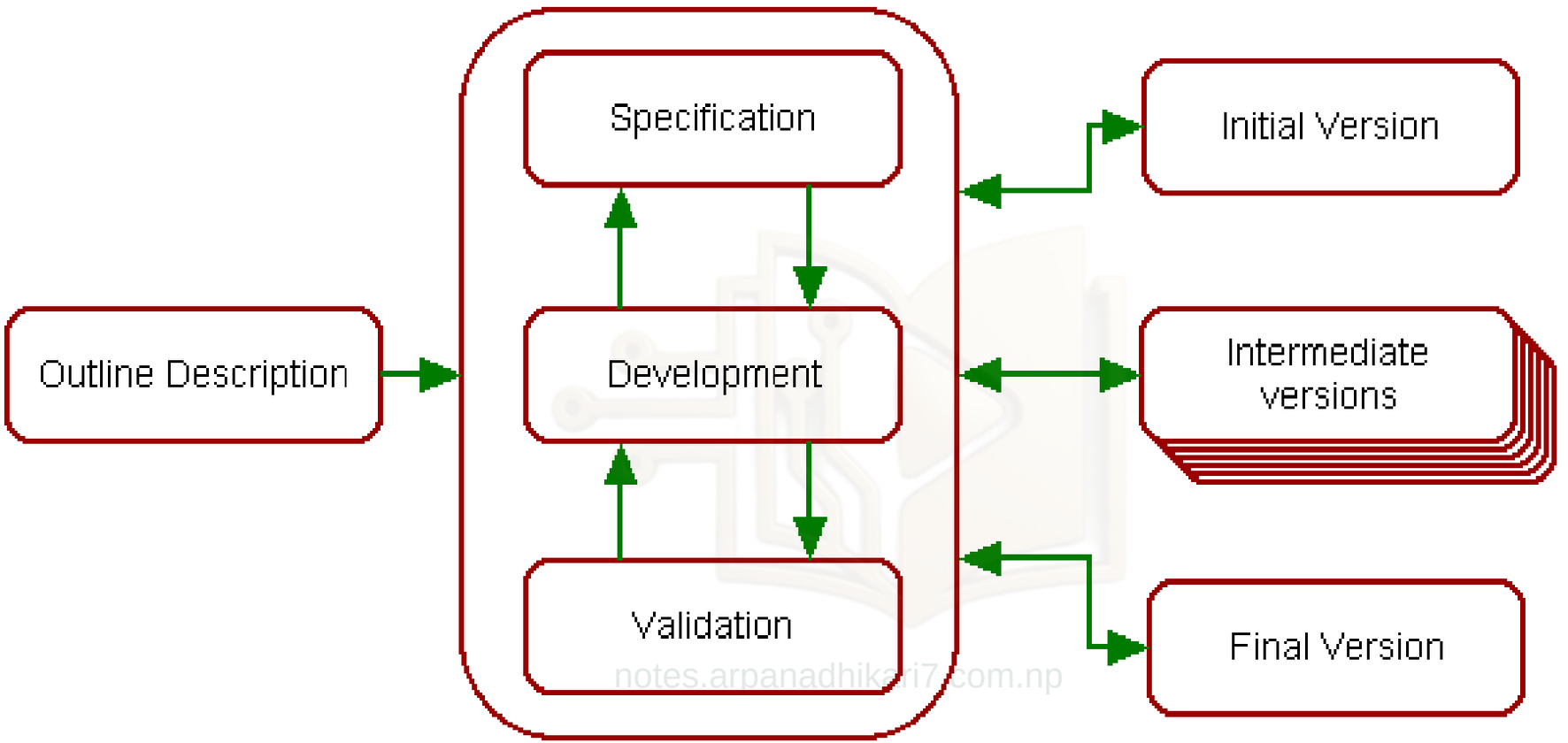


Figure: Evolutionary Model

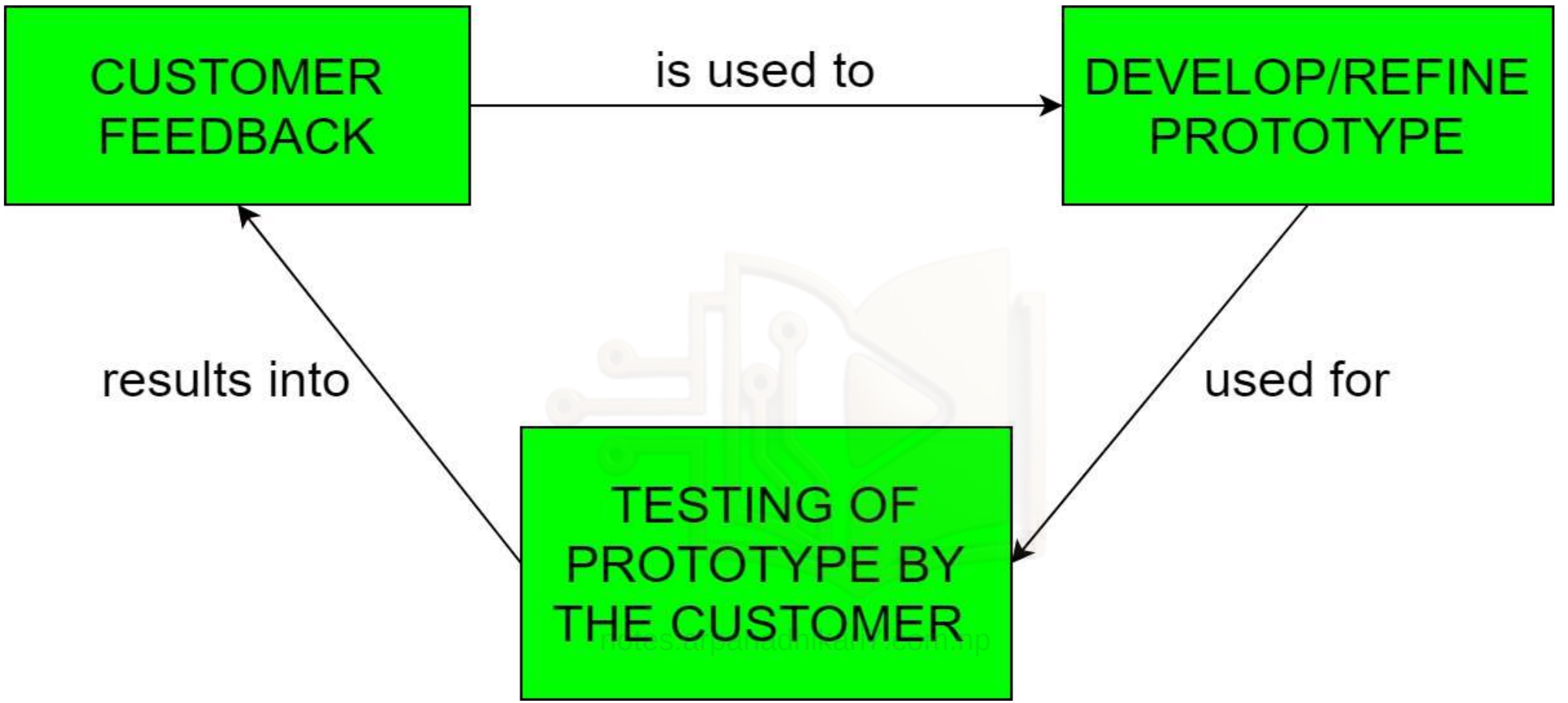
Available on notes.arpanadhikari7.com.np

Prototyping Model

- Prototype is a working model of software with some limited functionality.
- **Prototyping Model** is a software development model in which prototype is built, tested, and reworked until an acceptable prototype is achieved.
- This model is used when the customers do not know the exact project requirements beforehand.
- A prototype of the end product is first developed, tested and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



Available on notes.arpanadhikari7.com.np

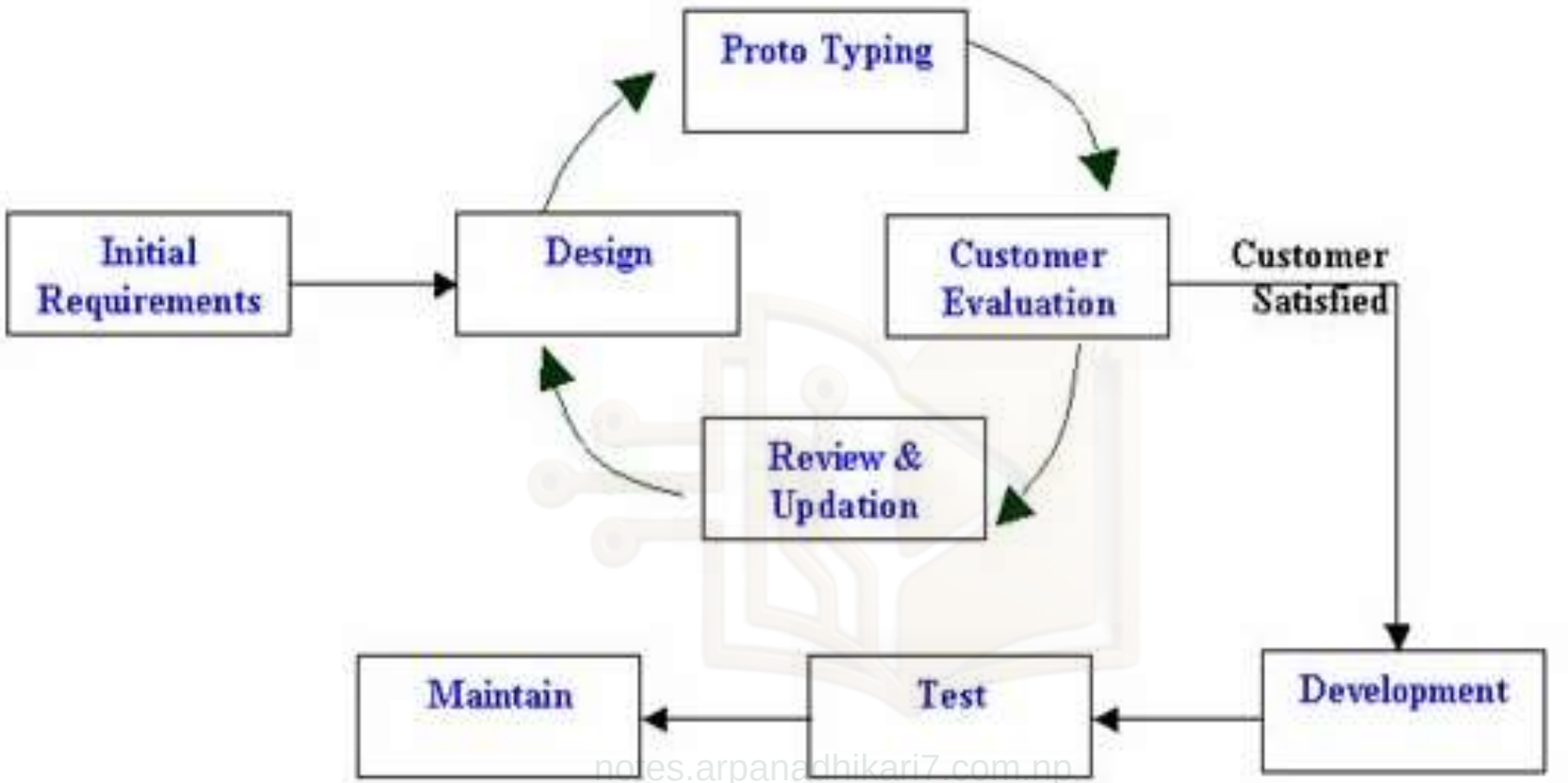


Figure: Prototyping Model

Available on notes.arpanadhikari7.com.np

When is prototyping model suitable?

- When the customers do not know the exact project requirements beforehand.
- Prototype model should be used when the desired system needs to have a lot of interaction with the end users.
- Excellent for designing good human computer interface systems.

notes.arpanadhikari7.com.np

Pros and Cons of Prototyping Model

Pros:

1. Increased user involvement in the product even before its implementation.
2. Since a working model of the system is displayed, the users get a better understanding of the system being developed.
3. Reduces time and cost as the defects can be detected much earlier.
4. Quicker user feedback is available leading to better solutions.
5. Missing functionality can be identified easily.
6. Confusing or difficult functions can be identified.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Cons:

1. Users may get confused in the prototypes and actual systems.
2. Practically, this methodology may increase the complexity of the system as scope of the system may expand beyond original plans.
3. Developers may try to reuse the existing prototypes to build the actual system, even when it is not technically feasible.
4. The effort invested in building prototypes may be too much if it is not monitored properly

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

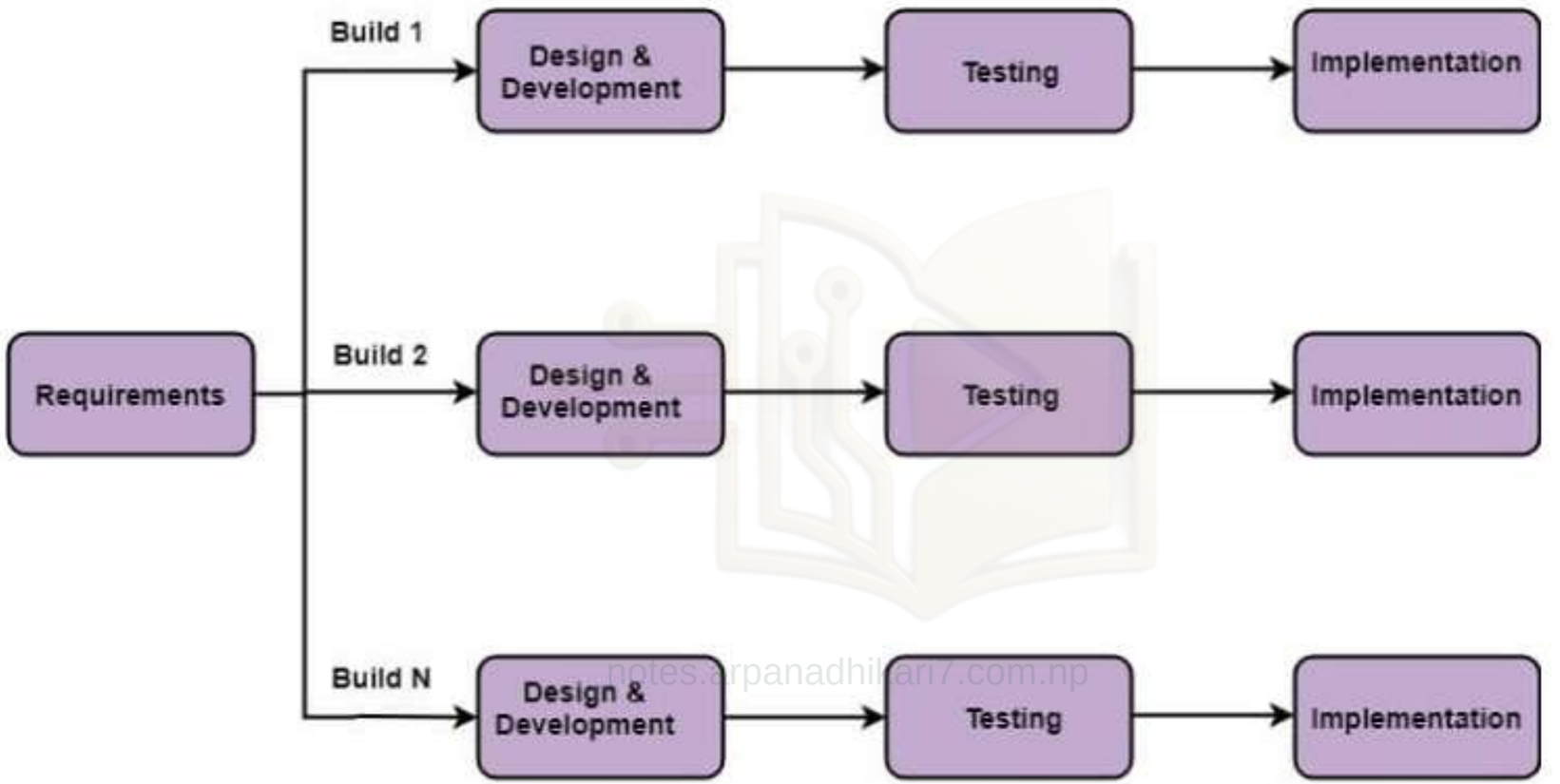
Incremental Development Model

- The **Incremental Model** is a software development approach where the project is divided into smaller, manageable increments or modules.
- Each increment represents a subset of the system's functionality, which is developed and delivered in successive iterations. The final system evolves as these increments are integrated, tested, and refined.
- Incremental: a little more is added each time/adding new functionality in small chunks
- It is a type of evolutionary model.
- Incremental Model is a process of software development where requirements are broken down into multiple standalone modules of software development cycle.
- Incremental development is done in steps from analysis design, implementation, testing/verification, maintenance.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Incremental Development Model



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

When is Incremental model suitable?

1. Such models are used where requirements are clear and can implement by phase wise.
2. Mostly such model is used in web applications and product based companies.

notes.arpanadhikari7.com.np

Pros:

1. Generates working software quickly and early during the software life cycle.
2. More flexible – less costly to change scope and requirements.
3. Easier to test and debug during a smaller iteration.
4. Easier to manage risk because risky pieces are identified and handled during its iteration.
5. Each iteration is an easily managed milestone.

Cons:

1. Each phase of an iteration is rigid and do not overlap each other.
2. Problems may arise pertaining to system architecture because not all requirements are gathered up front for the entire software life cycle.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

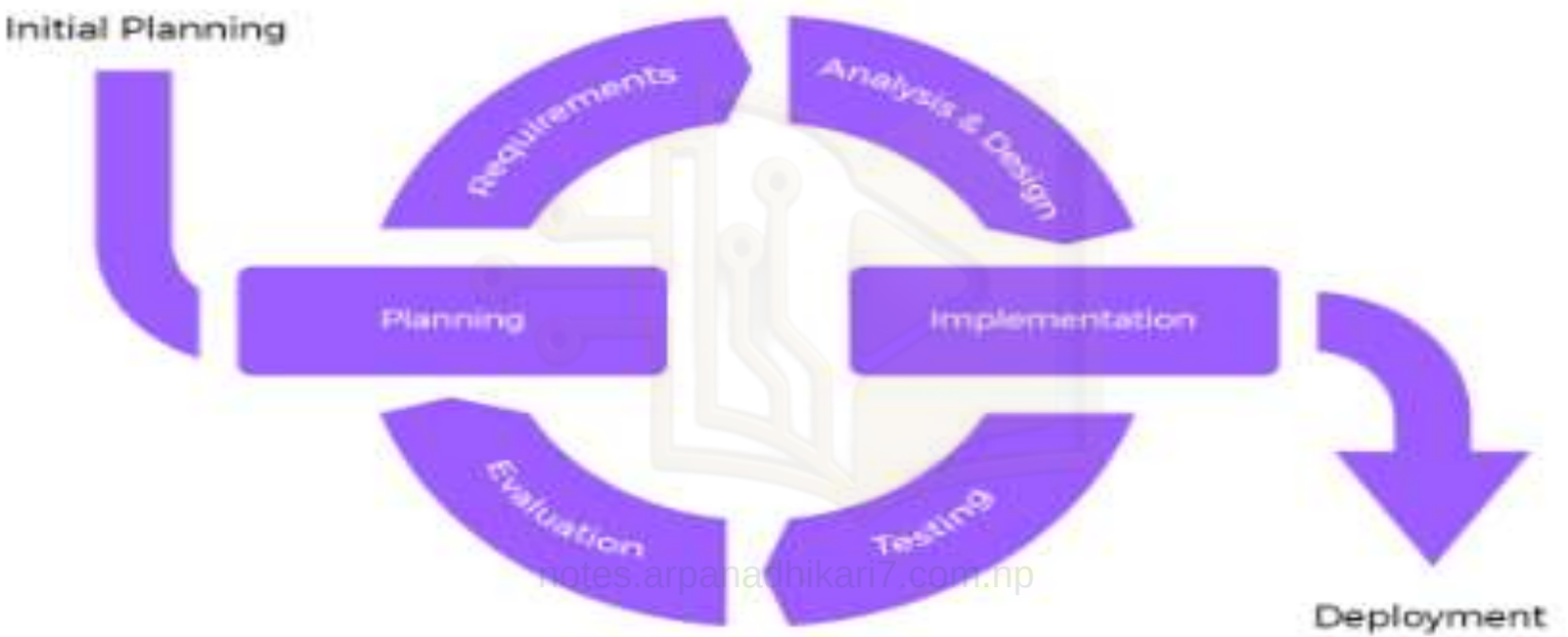
Iterative Model

- The **Iterative Model** is a software development approach where the system is built and refined through repeated cycles (iterations) rather than being developed in a single sequence. Each iteration involves planning, designing, coding, and testing, delivering a functional version of the software that evolves toward the final product.
- The Iterative Incremental Model is a smart way to design complex systems.
- Instead of trying to figure everything out at the beginning, this approach breaks the project into smaller parts. Each part is then developed, tested, and added to the system, bit by bit.
- This method is great for projects where things might change or if the requirements aren’t clear from the start.
- It allows designers to focus on making each part work well before adding more, leading to a better overall system.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Iterative Model



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Iterative Model

When to Use the Iterative Model

Unclear Requirements: When initial requirements are not well-defined or are expected to evolve

Large Projects: Suitable for complex systems requiring progressive refinement.

Risky Projects: Ideal for projects with high levels of technical or business risk.

Advantages:

I. Early Working Software: Provides a functional product early in development.
II. Risk Mitigation: Addresses critical issues in early stages.
III. Improved Requirements Understanding: Stakeholders can refine requirements over time.
IV. Continuous Testing: Enhances software quality by validating every iteration.

Disadvantages:

I. Resource-Intensive: Requires frequent testing and reviews, which can increase cost and time.
II. Complex Management: Planning and managing multiple iterations can be challenging.
III. Scope Creep: Continuous changes in requirements can lead to uncontrolled expansion of project scope.

notes.arpanadhikari7.com.np

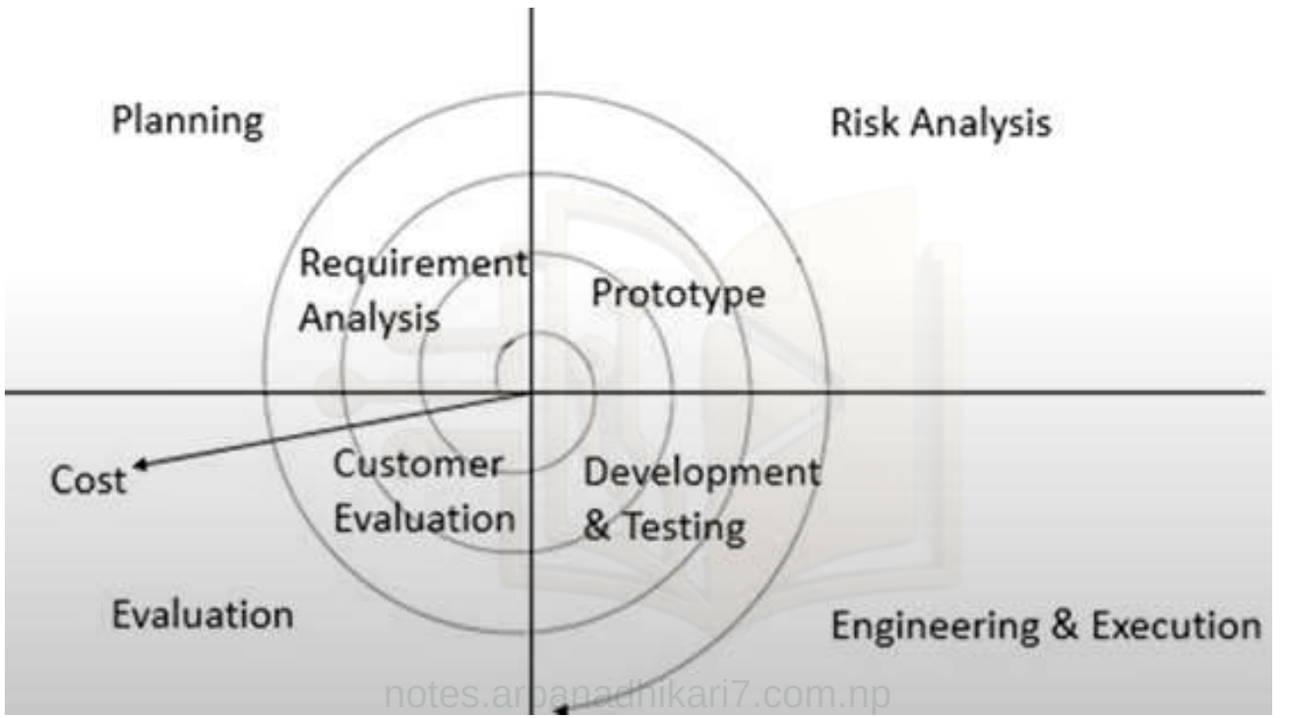
Available on notes.arpanadhikari7.com.np

Spiral Development Model

- Spiral model is a combination of iterative development process model and sequential linear development model i.e. the waterfall model with a very high emphasis on risk analysis.
- This model is best used for large projects which involves continuous enhancements.
- There are specific activities which are done in one iteration (spiral) where the output is a small prototype of the large software.
- The same activities are then repeated for all the spirals till the entire software is build.
- Each loop of the spiral is called a Phase of the software development process.
- The exact number of phases needed to develop the product depends upon the project risk.

Available on notes.arpanadhikari7.com.np

Spiral Development Model



notes.arpanadhikari7.com.np

Figure: Spiral Model

Available on notes.arpanadhikari7.com.np

Spiral Development Model

- Spiral model, generally involves four phases, which are followed repeatedly, in each round of the model, until no further requirements are needed to be implemented in the software product.
- The Radius of the spiral at any point represents the expenses(cost) of the project so far, and the angular dimension represents the progress made so far in the current phase.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

When is Spiral Model applicable?

- When there is a budget constraint and risk evaluation is important.
- For medium to high-risk projects.
- Long-term project commitment because of potential changes to economic priorities as the requirements change with time.
- Customer is not sure of their requirements which is usually the case.
- Requirements are complex and need evaluation to get clarity.
- New product line which should be released in phases to get enough customer feedback.
- Significant changes are expected in the product during the development cycle.

Available on notes.arpanadhikari7.com.np

Pros and Cons of Spiral Model

Pros:

1. Changing requirements can be accommodated.
2. Allows extensive use of prototypes.
3. Requirements can be captured more accurately.
4. Users see the system early.
5. Development can be divided into smaller parts and the risky parts can be developed earlier which helps in better risk management.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Pros and Cons of Spiral Model

Cons:

1. Management is more complex. Risk analysis requires high expertise.
2. End of the project may not be known early.
3. Not suitable for small or low risk projects and could be expensive for small projects.
4. Process is complex
5. Spiral may go on indefinitely.
6. Large number of intermediate stages requires excessive documentation.

notes.arpanadhikari7.com.np

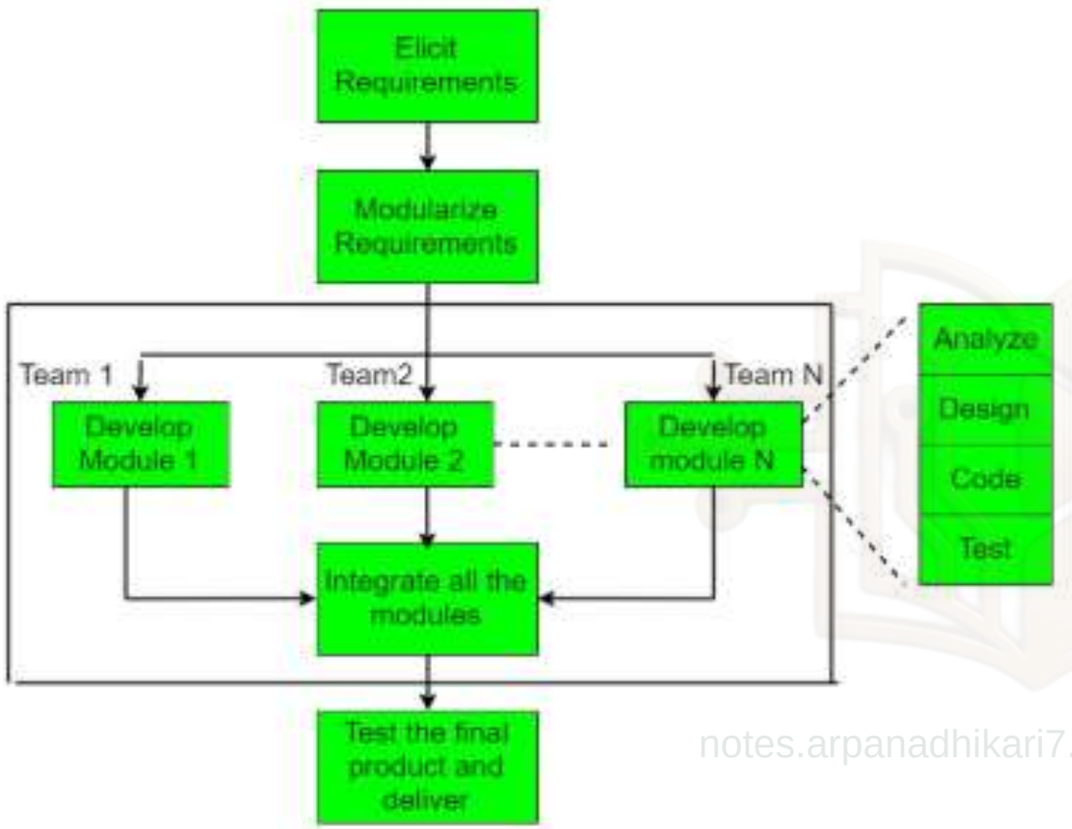
Available on notes.arpanadhikari7.com.np

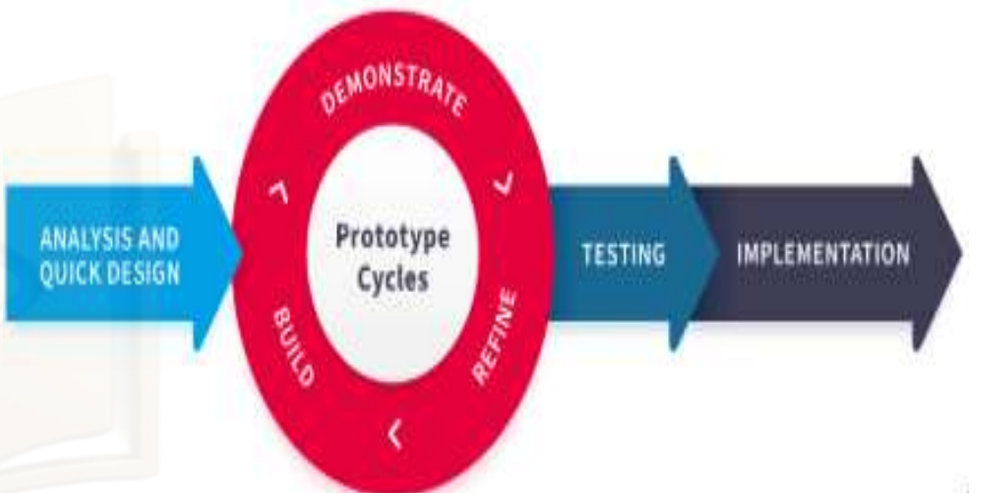
Rapid Application Development(RAD) Model

- It targets at developing software in a short span of time.
- It emphasizes on delivering projects in small pieces; the larger projects are divided into a series of smaller projects.
- The main features of RAD model are that it focuses on the reuse of templates, tools, processes, and code.

notes.arpanadhikari7.com.np

Rapid Application Development(RAD) Model





notes.arpanadhikari7.com.np

When to use RAD Methodology?

- When a system needs to be produced in a short span of time (2-3 months)
- When the requirements are known
- When the user will be involved all through the life cycle
- When technical risk is less
- When there is a necessity to create a system that can be modularized in 2-3 months of time
- When a budget is high enough to afford designers for modeling along with the cost of automated tools for code generation

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Pros and Cons

Pros:

- Reduced development time.
- Increases reusability of components
- Quick initial reviews occur
- Encourages customer feedback
- Integration from very beginning solves a lot of integration issues.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Cons:

- Depends on strong team and individual performances for identifying business requirements.
- Only system that can be modularized can be built using RAD
- Requires highly skilled developers/designers.
- High dependency on modeling skills
- Inapplicable to cheaper projects as cost of modeling and automated code generation is very high.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Agile Development Model

- Agile is the ability to create and respond to change.
- It is a way of dealing with, and ultimately succeeding in, an uncertain and turbulent environment.
- The Agile model was primarily designed to help a project to adapt to change requests quickly. So, the main aim of the Agile model is to facilitate quick project completion.
- Agility is achieved by fitting the process to the project, removing activities that may not be essential for a specific project. Also, anything that is wastage of time and effort is avoided.
- Agile approaches to software development consider design and implementation to be the central activities in the software process.
- Specification, Design, Implementation and testing are interleaved.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Agile Development Model

- Agile development approaches evolved in the 1990s as a reaction to documentation and bureaucracy-based processes, particularly the waterfall approach.
- Agile approaches are based on some common principles, some of which are:
  I. Working software is the key measure of progress in a project.
  II. For progress in a project, therefore software should be developed and delivered rapidly in small increments.
  III. Even late changes in the requirements should be entertained.
  IV. Face-to-face communication is preferred over documentation
  V. Continuous feedback and involvement of customers are necessary for developing good-quality software.
  VI. A simple design that involves and improves with time is a better approach than doing an elaborate design up front for handling all possible scenarios.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Agile Principles

1. Customer Involvement: Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
2. Incremental Delivery: The software is developed in increments with the customer specifying the requirements to be included in each increment.
   - Frequent delivery of working software
3. People not process: The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes
   - Support, trust, and motivate the people involved
4. Embrace Change: Expect the system requirements to change and so design the system to accommodate these changes.
5. Maintain simplicity: Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Agile Principles

6. Enable face-to-face interactions
7. Working software is the primary measure of progress
8. Agile processes to support a consistent development pace
9. Attention to technical detail and design enhances agility
10. Regular reflections on how to become more effective
11. Customer satisfaction through early and continuous software delivery

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Existing Agile Development Methodologies

- Extreme Programming (XP)
- Scrum
- Agile Project Management and Scaling

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Extreme Programming(XP)

- **Extreme Programming (XP)** is a popular Agile software development methodology designed to improve software quality and responsiveness to changing customer requirements.
- XP emphasizes a close working relationship between the development team, the customer, and stakeholders, with an emphasis on rapid, iterative development and deployment.

notes.arpanadhikari7.com.np

Extreme Programming(XP)

- There are five core values of Extreme Programming (XP):

Communication: The essence of communication is for information and ideas to be exchanged amongst development team members so that everyone has an understanding of the system requirements and goals. Extreme Programming (XP) supports this by allowing open and frequent communication between members of a team.

Simplicity: Keeping things as simple as possible helps reduce complexity and makes it easier to understand and maintain the code.

Feedback: Feedback loops which are constant are among testing as well as customer involvements which helps in detecting problems earlier during development.

Courage: Team members are encouraged to take risks, speak up about problems, and adapt to change without fear of repercussions.

Respect: Every member's input or opinion is appreciated which promotes a collective way of working among people who are supportive within a certain group.

Available on notes.arpanadhikari7.com.np

Extreme Programming(XP)

- The Extreme Programming Life Cycle consist of five phases:

Planning: The first stage of Extreme Programming is planning. During this phase, clients define their needs in concise descriptions known as user stories. The team calculates the effort required for each story and schedules releases according to priority and effort.

Design: The team creates only the essential design needed for current user stories, using a common analogy or story to help everyone understand the overall system architecture and keep the design straightforward and clear.

Coding: Extreme Programming (XP) promotes pair programming i.e**.** developers work together at one workstation, enhancing code quality and knowledge sharing. They write tests before coding to ensure functionality from the start and frequently integrate their code into a shared repository with automated tests to catch issues early.

Testing: Extreme Programming (XP) gives more importance to testing that consist of both unit tests and acceptance test**.** Unit tests, which are automated, check if specific features work correctly. Acceptance tests, conducted by customers, ensure that the overall system meets initial requirements. This continuous testing ensures the software's quality and alignment with customer needs.

Listening: In the listening phase regular feedback from customers to ensure the product meets their needs and to adapt to any changes.

Available on notes.arpanadhikari7.com.np

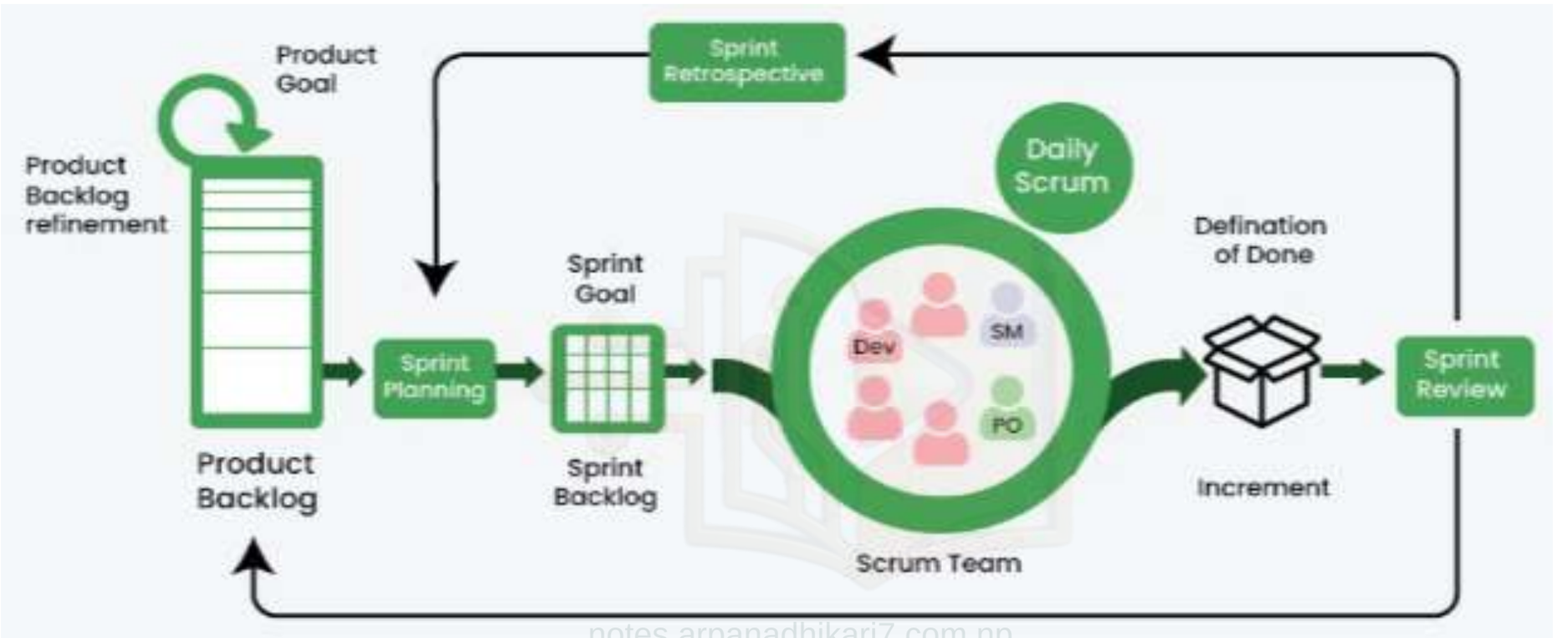
Scrum

- Scrum is an **agile framework** for managing and completing complex software development projects.
- It emphasizes teamwork, collaboration, adaptability, and iterative progress toward a well-defined goal.
- Scrum enables teams to deliver high-value products efficiently while responding effectively to changing requirements.
- **Silent features of Scrum**
  - Scrum is a light-weighted framework
  - Scrum emphasizes self-organization
  - Scrum is simple to understand
  - Scrum framework helps the team to work together

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Scrum



notes.arpanadhikari7.com.np

Scrum

Key Components of Scrum

1. Roles

- **Product Owner**:
  - Represents the stakeholders and customers.
  - Manages the **Product Backlog** (a prioritized list of tasks or features).
- **Scrum Master**:
  - Facilitates the Scrum process and removes obstacles for the team.
  - Ensures the team adheres to Scrum principles.
- **Development Team**:
  - A self-organizing group responsible for delivering the product increment.
  - Typically includes cross-functional members (developers, testers, etc.).

Available on notes.arpanadhikari7.com.np

Scrum

2. Artifacts

- **Product Backlog**:
  - A prioritized list of all features, enhancements, bug fixes, and tasks required in the product.
- **Sprint Backlog**:
  - A subset of the product backlog selected for implementation during a specific sprint.
- **Increment**:
  - A working, shippable piece of the product delivered at the end of a sprint.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Scrum

3. Events

- **Sprint**:
  - A time-boxed iteration, typically lasting 1–4 weeks, where the team works on a specific set of tasks.
- **Sprint Planning**:
  - A meeting at the start of a sprint to define the sprint goal and plan the sprint backlog.
- **Daily Scrum (Stand-Up)**:
  - A short daily meeting (15 minutes) where team members discuss:
    - What they did yesterday.
    - What they will do today.
    - Any obstacles in their way.
- **Sprint Review**:
  - Held at the end of a sprint to demonstrate the completed work to stakeholders.
- **Sprint Retrospective**:
  - A meeting to reflect on the sprint and identify improvements for the next sprint.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Scrum

3. Events

- **Sprint**:
  - A time-boxed iteration, typically lasting 1–4 weeks, where the team works on a specific set of tasks.
- **Sprint Planning**:
  - A meeting at the start of a sprint to define the sprint goal and plan the sprint backlog.
- **Daily Scrum (Stand-Up)**:
  - A short daily meeting (15 minutes) where team members discuss:
    - What they did yesterday.
    - What they will do today.
    - Any obstacles in their way.
- **Sprint Review**:
  - Held at the end of a sprint to demonstrate the completed work to stakeholders.
- **Sprint Retrospective**:
  - A meeting to reflect on the sprint and identify improvements for the next sprint.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Scrum

Workflow in Scrum

1. The **Product Owner** prioritizes the product backlog.
2. During **Sprint Planning**, the team selects items from the backlog to work on in the sprint.
3. The team works on the sprint backlog during the sprint, holding daily scrums to stay aligned.
4. At the end of the sprint:
    1. The team demonstrates the increment in a **Sprint Review**.
    2. Reflects on the sprint in a **Sprint Retrospective**.
5. The cycle repeats, continuously delivering increments until the product is complete.

Available on notes.arpanadhikari7.com.np

Scrum

Advantages of Scrum

1.Adaptability:

Responds quickly to changes in requirements or priorities.

2.Faster Delivery:

Enables frequent delivery of functional product increments.

3.Improved Collaboration:

Encourages regular communication among team members and stakeholders.

4.Focus on Value:

Prioritizes features that deliver the most business value.

5.Continuous Improvement:

Retrospectives ensure teams learn and improve with each sprint.

Available on notes.arpanadhikari7.com.np

Scrum

Disadvantages of Scrum

- Requires a self-organizing and disciplined team for success.
- New teams may struggle with adopting Scrum practices effectively.
- Scrum framework do not allow changes into their sprint.

notes.arpanadhikari7.com.np